

Draft

Version : 0.2
Date : 26th Jan 2026
Authors : Prabhat Karlekar
Department :

1. Background

The UNFCCC (United Nations Framework Convention on Climate Change) formalized Clean Development Mechanism (CDM) under the Kyoto Protocol in 1997. Carbon credits emerged as an economic instrument to price carbon emissions and address global warming through economic mechanisms rather than regulation. Policymakers introduced cap-and-trade and offset mechanisms so emissions reductions could be quantified, verified, and converted into tradable units. The idea was One carbon credit equal to 1 tonne of CO₂-equivalent reduced, avoided, or removed.

During the expansion of voluntary era (post 2010) companies like Delta, Disney bought Voluntary Credits to look good (CSR) and phantom credits were sold for forests that were never in danger, or re-sold multiple times. In CDM, credits were treated as certificates, not assets which has reliance on manual audits with high costs, slow issuance, broker dominance and bureaucratic. Post-Kyoto the demand collapsed, prices were crashed and small players got excluded.

Standards like Verra, Gold Standard (GS) emerged for corporate CSR digitized the trading, nature-based credits (forestry) surged. The market was plagued by fragmented registries, phantom credits (issued for non-existent reductions) and double-counting (same credit claimed by multiple entities). At 2015 world agreed under Paris Agreement (Article 6) that carbon trading must be high integrity and every tonne must be accounted for. The focus shifts on integrity, transparency, and avoidance of double counting. Now, carbon credits are becoming regulated financial environmental instruments with infrastructure and not just markets. Infrastructure to verify small, decentralized projects cheaply and securely does not exist.

India is the 3rd largest emitter but also has the biggest potential for low-cost reduction, 2023 onwards India officially announce Carbon Credit Trading Scheme (CCTS) and its own domestic carbon market (ICM). India announced a domestic, sovereign carbon market under law, Energy Conservation (Amendment) Act, 2022, Carbon Credit Trading Scheme (CCTS), 2023 with compliance-first market, domestic use priority, government-controlled registry as key characteristics. Now, a farmer planting trees can sell credits to a Steel Plant that pollutes, legal framework exists, but the technology to verify millions of small farmers cheaply does not.

This creates need for a new Digital Public Infrastructure (DPI) for carbon credits capable of programmatic verification, cryptographic integrity, and lifecycle enforcement, ensure privacy and scale to meet the demands of such economy.

2. Proposed National Carbon Credit Platform Design

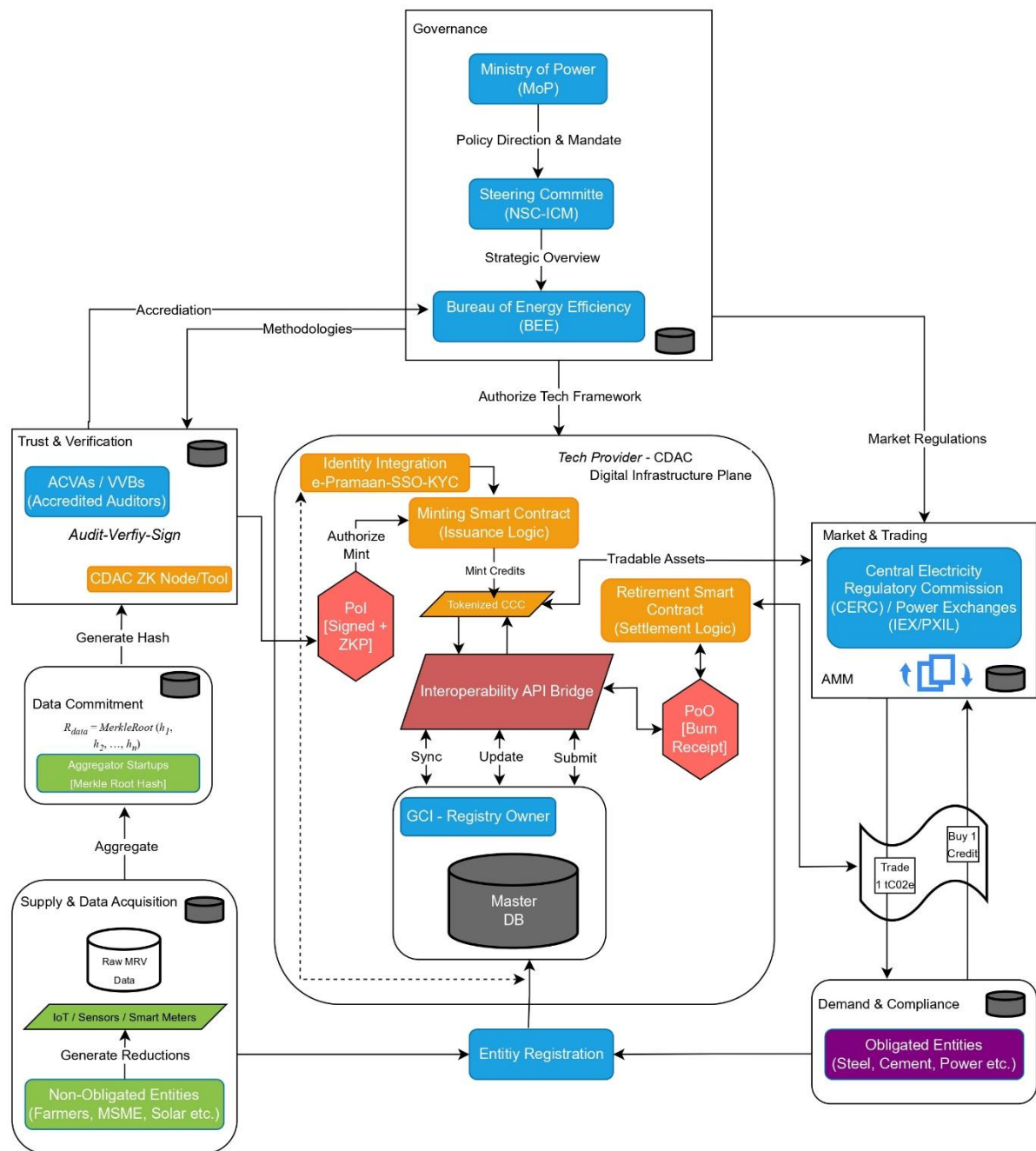


Figure 1: Proposed Framework

Design Document

Privacy-Preserving Decentralized Carbon Credit Framework

Version : 0.2
Date : 26th Jan 2026
Authors : Prabhat Karlekar
Mentors :
Department :

The design addresses the challenges in existing carbon markets, incorporating the Two-Proof Model (Proof-of-Integrity and Proof-of-Offset).

1. Participants

This system preserves collaborative ecosystem among institutions while making their actions cryptographically accountable:

Participant / Entity	Role	Responsibilities
Ministry of Power (MoP) & National Steering Committee (NSC-ICM)	Governance, Policy Owner & Oversight	- Sets sector specific emission targets, approves methodologies - NSC oversight inter-institutional coordination
Standards / Regulators - Bureau of Energy Efficiency (BEE)	Administrator, define rules & methodologies	- Create methodologies - Whitelists Accredited Carbon Verification Agencies (ACVAs) - Issue Carbon Credit Certificates (CCCs)
Accredited Carbon Verification Agencies (ACVAs)	Audit / verify data & calculations	- Auditors accredited by BEE - Sign attestations, ZK proofs - Submits Proof-of-Integrity(PoI) to mint credits
Grid Controller of India (GCI)	Registry Owner & Operator	- Maintains ownership and settlement records - Operates national carbon registry database
----	Digital Infrastructure Provider	- Identity Integration () - Develops DPI plane and ZK verification Nodes

		- Interoperability bridge to GCI registry
Aggregators Startups	Data Aggregation	- Hash data points - Create Merkle root
Non-Obligated Entities (e.g., farmers, MSME, Solar)	Sellers	- Generate measurable reductions/removals or absorption - Submit data for minting credits
Obligated Entities (e.g., steel, cement, power, etc.)	Buyers / Compliance buyers	- Purchases credits - Burn credits to generate Proof-of-Offset(PoO)
Liquidity Providers / Market Makers	Price discovery	- Earns fees from trading.

2. Design and Architecture

The system is a decentralized marketplace for minting, trading, and retiring tokenized carbon credits, bridging voluntary and compliance markets:

2.1 High-Level System Design

Retail / P2P Micro-Offset Apps: Enable individual users (e.g., citizens or small businesses) to engage in peer-to-peer trading. Users submit offset claims here, triggering PoO upon retirement.

Dashboards: Web-based interface for clients to track large-scale emissions, facilitate bulk transactions, manage accounting, and generate verifiable offset and reports. Credits can be swapped with stablecoins to address liquidity and hoarding issues.

Carbon Registry (Decentralized) : A registry and marketplace implemented via smart contracts for lifecycle management (mint, trade, burn). Credits are minted only after successful Proof-of-Integrity (PoI). Credits are irreversibly retired through burn events, producing a Proof-of-Offset (PoO).

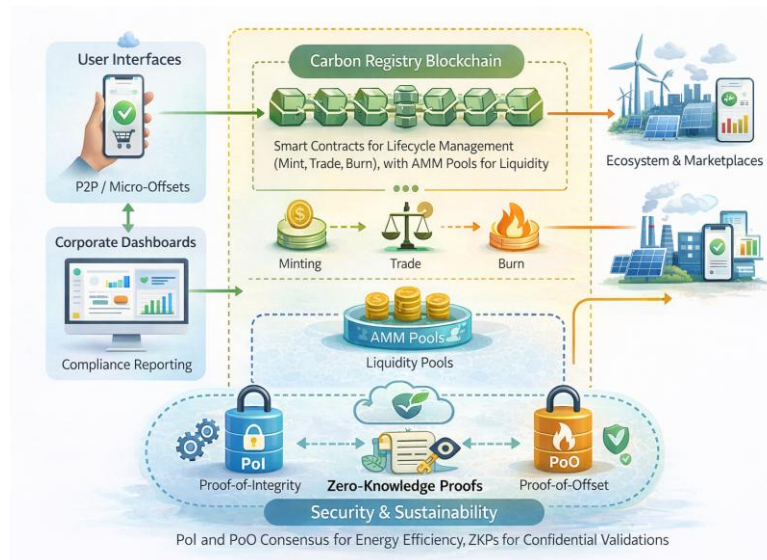


Figure 2: High-Level Design

AMM Pools (Liquidity): To ensure there is always a buyer or seller available, the system uses Automated Market Maker (AMM) pools. These liquidity pools allow for instant trades at algorithmically determined prices.

Ecosystem & Marketplaces: The registry connects outward to the real economy, interoperable with external marketplaces as credits remain serialized, traceable and non-duplicate.

Security & Sustainability: This layer enforces the "Two-Proof Model" (PoI and PoO) via low-energy consensus and uses ZKPs for confidential validations:

- **Proof-of-Integrity(PoI):** Governs credit creation and existence. Issued by authorized entities following an approved methodology, it prevents fake credits or over-issuance as it is enforced before minting.
- **Proof-of-Offset (PoO):** Guarantees credit consumption and settlement. It ensures credits existence, are not double-spent, was valid at the time of use and are permanently taken out of circulation at the time of retirement.

2.2 High-Level System Architecture

The system architecture is designed, ensuring verification, registry enforcement, market operations, and application logic. This enables trading without altering credits, and prevents double-counting, supports ZKPs for privacy-preserving validations as the backbone.

2.2 High-Level System Architecture

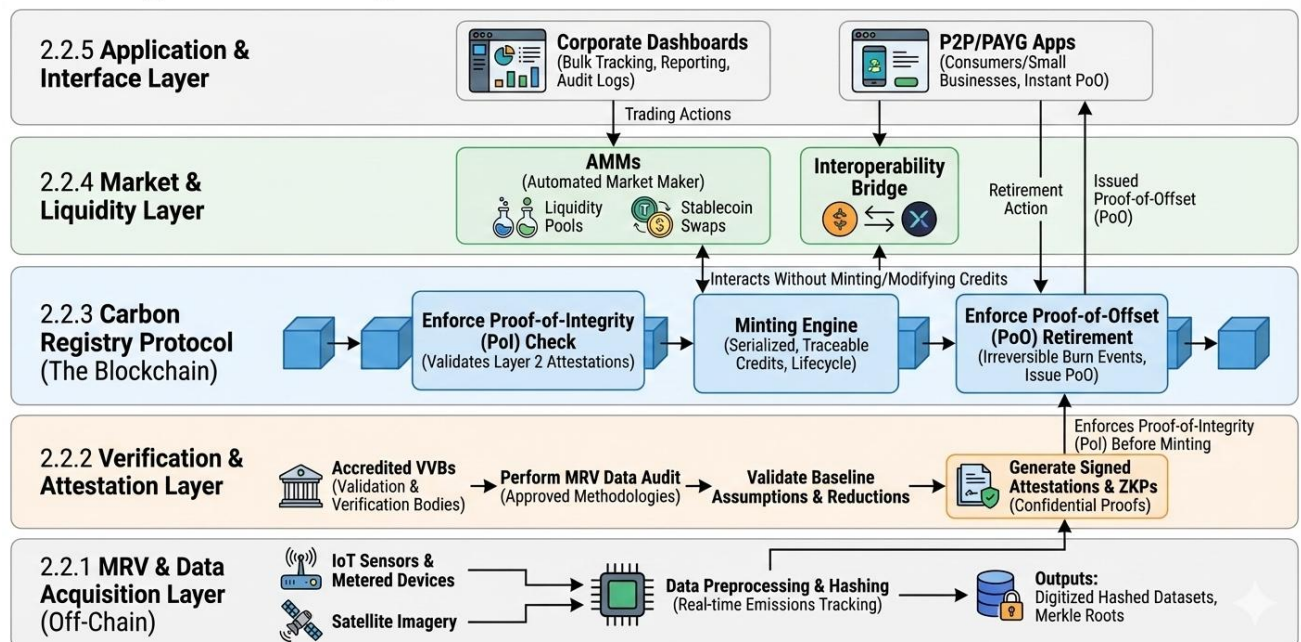


Figure 3: High-Level System Architecture

2.2.1 MRV & Data Acquisition Layer (Off-Chain):

This layer responsible for capturing and preprocessing (measurement, reporting, and verification) Data, encompasses real-world sources of carbon emissions reduction activities to ensure accuracy before it touches the registry:

- Sources: IoT sensors and metered devices, satellite imagery for large-scale monitoring for real-time emissions tracking.
- Outputs: Digitized hashed datasets, Merkle roots, and summarized metrics.

2.2.2 Verification & Attestation Layer:

In this layer, data is audited and validated by accredited Validation and Verification Bodies (VVBs), include:

- Perform MRV data audit against approved methodologies.
- Validate baseline assumptions and reductions which generates signed attestations and ZKPs for confidential proofs

As Cryptographically signed attestations ready for the registry, this layer enforces Proof-of-Integrity (PoI) before minting, preventing over-issuance or fakes.

2.2.3 Carbon Registry Protocol:

The immutable ledger responsible for the "Two-Proof" enforcement which handles credit existence and not trading:

- Enforce Proof-of-Integrity (PoI): Validates Layer 2 attestations before allowing any minting.
- Minting Engine: Mint serialized, traceable (ownership, transfers, expiry/decay rules), a complete lifecycle of carbon credits.
- Enforce Proof-of-Offset (PoO): Handle retirement, generate irreversible retirement (burn) events for verifiable claims, produce Proof-of-Offset (PoO).

All credits exist only once in this layer.

2.2.4 Market & Liquidity Layer:

The financial layer where price discovery occurs, interacts with Layer 3 without minting/modifying credits:

- AMMs (Automated Market Maker): Retail/voluntary liquidity (e.g., Uniswap-like pools with stablecoin swaps to deter hoarding).
- Interoperability: Connects to external marketplaces (e.g., traditional exchanges) while maintaining serialization.

Markets cannot mint, modify, or invalidate credits. They operate strictly on registry-issued assets.

2.2.5 Application & Interface Layer:

User-facing interfaces for interaction and reporting built atop the registry and markets to exposes the system to users and institutions

- Corporate Dashboards: Dashboards for bulk tracking, reporting, audit logs and swaps.
- P2P/PAYG: Consumers or small businesses trading with instant PoO on retirement.

All offset claims generated here are finalized only after Proof-of-Offset (PoO) is issued by the registry.

3. Data Structure

To enforce the "Two-Proof Model" data structures are designed for immutability and traceability, these structures embed provenance (PoI) and settlement proofs (PoO), both are zk-SNARK proofs (compact, verifiable), ensuring privacy and efficiency.

- Proof-of- Integrity (PoI): Consolidated zk-SNARK for pre-mint validation, generate off-chain by the VVB and verified on-chain before minting acts as the "Minting Permit".

Proof-of-Integrity Object:

```
{
  "proof_type": "PoI",
  "inputs": {
    "project_id": "PRJ-123",                // Links to the registered Project Developer
    "project_status": "REGISTERED", // REGISTERED / ACTIVE / SUSPENDED /
    TERMINATED
    "mrv_mode": "CONTINUOUS", // CONTINUOUS / PERIODIC / EVENT_BASED
    "timestamp": "2026-01-01T12:00:00Z",    // Time of issuance
    "vvb_signature": "0x7a8f...", // Cryptographic signature of the accredited VVB
    "methodology_hash": "SHA256(VM0042)",    // Compliance reference
    "data_root": "0xMerkleRoot...", // Reduction proof (hashed MRV)
    "serialization_range": "0-1000000" // Unique credit IDs
    "amount_tCO2e": 1, // tCO2e removed/avoided
    "jurisdiction": "IN",
    "market_scope": "VCM", // ETS / VCM
  },
  "output": {
    "credit_id": "CC-789",
    "valid_from": "2026-01-01T00:00:00Z",
    "valid_until": "2026-12-31T23:59:59Z"
    "zk_proof": "0x123abc...", // Verifiable without raw data
    "cc_mint_amount": 1, // Carbon Credits (tCO2e) authorized to be minted
    "owner": "0xWalletAddress..", // Wallet/DID of the owner
    "status": "VALID" // State: VALID / EXPIRED / REVOKED
  }
}
```

- Proof-of- Offset (PoO): Generated upon retirement of credits and non-transferable "Settlement Receipt".

Proof-of- Offset (PoO) Object:

```
{
  "proof_type": "PoO",
  "inputs": {
    "project_id": "PRJ-123",           // Links to the registered Project Developer
    "credit_id": "CC-789",
    "serialization_range": "0-1000000" // Unique IDs
    "market_scope": "VCM",           // ETS / VCM
    "cc_amount": 1
  },
  "output": {
    "burn_tx_hash": "0x998877...",    // non-double-counting proof
    "timestamp": "2026-01-01T12:00:00Z", // Time of retirement
    "beneficiary": "Corporate_Entity_X", // Who gets the credit?
    "owner": "0xWalletAddress..",      // Wallet/DID of the owner
    "claim_type": "Corporate_NetZero",  // Corporate_NetZero / PAYG / Compliance
    "zk_proof": "0x123abc...",         // Verifiable without raw data
    "amount_tCO2e": 1,                 // tCO2e authorized to be emit
    "status": "FINALIZED"              // State: REJECTED / FINALIZED
  }
}
```

- Project Object: Registry metadata for governance, details about the project.

```
{
  "registration_data": {
    "project_id": "PRJ-123",           // Unique Identifier
    "owner": "0xWalletAddress..",      // Wallet/DID of the owner
    "project_name": "Solar Phase 1",
    "sector": "Renewable_Energy",      // e.g., Forestry, DirectCapture
    "methodology_id": "VM0042",        // Verra/GoldStandard Code/ETS
    "geography": "IN-MH",
    "market_scope": "VCM",             // ETS / VCM
    "mrv_mode": "PERIODIC",
    "verification_cycle": "Annual"     // Expected audit frequency
    "baseline_hash": "0xabc123...",    // Hash of the baseline data
    "leakage_risk": "Medium",          // Risk buffer calculation input
  },
  "status": "REGISTERED"               // PENDING, REGISTERED, SUSPENDED
}
```

- Carbon Credit Token Object:

```
{
  "credit_id": "#001",                // Token ID
}
```

```

"project_id": "PRJ-123",
"vintage_year": "2025",
"amount_tCO2e": 1000,
"cc_amount": 1
"serialized_range": "0-1000000",
"credit_type": "Removal"
"owner": "0xWalletAddress...",
"jurisdiction": "India",
"market_scope": "VCM",
"expiry_date": "2032-12-31",
"status": "ACTIVE"
}

```

// Link to Project Object
// Emission year (critical for pricing)

// Removal / Avoidance / Emit
// Current holder
// Origin Country
// ETS / VCM
// Automated decay trigger
// ACTIVE / RETIRED / EXPIRED

4. Workflow

This section details the step-by-step execution of the protocol, implements the credit lifecycle as a state machine. Each transition is enforced by smart contracts and cryptographic proofs. The complete workflow is explained in the steps below and illustrated in the sequence diagram (Fig. 3).

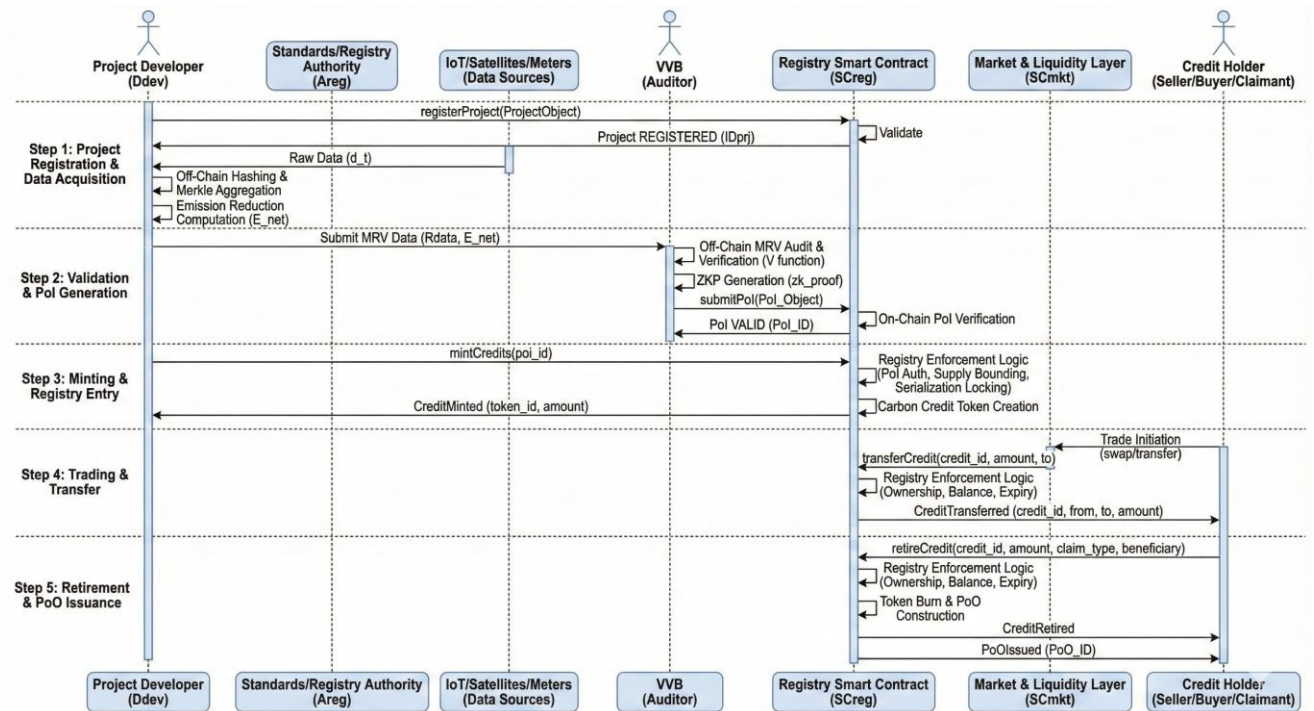


Figure 4: Sequence Diagram

4.1 Project Registration & Data Acquisition

Actors Involved:

- Project Developer (D_{dev}): Initiates registration and data capture (IoT/satellites/meters).
- Standards/Registry Authority (A_{reg}): Establishes, approves methodologies, scope and whitelist projects.

4.1.1 Project Initialization (On-Chain)

Before any emissions data is accepted, the D_{dev} must register the project entity on the Registry Smart Contract SC_{reg} (on-chain).

On-Chain Action $\rightarrow D_{dev}$ calls `registerProject(ProjectObject)`

The smart contract enforces `methodology_id` existence, `market_scope` compatibility and developer authorization (whitelisting).

To prevent duplicate or spoofed registrations, the unique Project ID (ID_{prj}) is derived deterministically:

$$ID_{prj} = H(Wallet_{owner} \parallel Sector \parallel Geography \parallel Nonce) \quad (1)$$

and Updates "status": "REGISTERED".

4.1.2 Baseline Establishment (Off-Chain)

The baseline represents counterfactual emissions (*what would have happened without the project*), $E_{baseline}(t)$ calculated using the approved methodology (e.g., VM0042):

$$E_{baseline}(t) = Historical_{avg} \times Geography \times Emission_{factor} \quad (2)$$

Raw baseline data is never published / stored, only the hash (immutable reference) is stored on-chain in the Project Object: "baseline_hash" = $H(\text{baseline_dataset})$

4.1.3 Data Acquisition & Merkle Aggregation (Off-Chain)

Sources capture actual emissions/reductions data (d_i) based on "mrv_mode":

- Raw Data Stream: Record data points $d_1, d_2, \dots, d_n$ at interval t .
- Hashing: Each data point is hashed: $h_i = H(d_i \parallel \text{timestamp})$.
- Root Generation (Merkle Aggregation): Compact data integrity root, proof of multiple points:

$$R_{data} = \text{MerkleRoot}(h_1, h_2, \dots, h_n) \quad (3)$$

This R_{data} will later become the “data_root” field in the PoI, allowing the auditor to verify readings via a single hash.

4.1.4 Emission Reduction Computation

The system computes the net reduction (E_{net}), using approved methodology and $E_{measured}$ (t), actual measured emissions which determines the potential credit amount:

$$\Delta CO_2(t) = E_{baseline}(t) - E_{measured}(t) \quad (4)$$

$$E_{net} = \Delta CO_2(t) \times (1 - L - U) \quad (5)$$

Where,

- L : Leakage deduction factor (from leakage_risk in Project Object).
- U : Uncertainty buffer.

This net reduction is the value later asserted inside PoI, "amount_tCO2e": X

4.2 Validation & Proof-of-Integrity (PoI) Generation

This step bridges the physical-world evidence (Off-Chain) and the Digital Ledger (On-Chain).

Actors Involved:

- Project Developer (D_{dev}): Provides read-only access to off-chain MRV data (R_{data}).
- Validation & Verification Body (VVB): Accredited independent third-party auditor (e.g., TÜV, SGS, KPMG) responsible for the off-chain audit.
- Carbon Registry Smart Contract (SC_{reg}): Enforces PoI verification on-chain, that validates the PoI signature and VVB whitelist status.

PoI Preconditions:

Requirement	Source	Description
Project status	Step 1	Must be REGISTERED on-chain.
Approved methodology	Registry	Must be whitelisted (e.g., VM0042)
Baseline Hash	Project Object	Must match the project’s initial registration.
MRV data root (R_{data}) available	Step 1	Merkle root (R_{data}) must be finalized off-chain.
VVB whitelisted	Registry Authority	-

If any condition fails, PoI generation is rejected.

4.2.1 MRV Audit by VVB (Off-Chain)

The VVB independently audits the project's MRV data using Merkle proofs (no raw is data expose). VVB executes a verification function V to ensure the data supports the claim:

$$V(R_{data}, E_{baseline}, Methodology) \rightarrow \{True, False\}$$

Checks,

- Baseline Correctness: Does baseline_hash matches ?
- Measurement Integrity: Do the raw data points hash to R_{data} ?
- Methodology Compliance: Are the reduction formulas applied correctly ?

4.2.2 Independent Reduction Verification

VVB recomputes the emissions reduction independently to confirm the claim using eq. 4 and eq. 5.

Validity Condition:

$$E_{net} > 0 \text{ (additionality)}$$

If false $\rightarrow$ PoI is rejected.

4.2.3 Zero-Knowledge Proof (zk-SNARK) Construction

To preserve confidentiality of proprietary industrial data, the VVB generates a zk-SNARK proving compliance without revealing the raw dataset D : “Given baseline B , MRV data D , and methodology M , the claimed reduction E_{net} is valid.”

Zero-Knowledge Statement:

$$\exists(B, D) : \begin{cases} H(B) = baseline_hash \\ MerkleRoot(D) = R_{data} \\ Reduction(B, D, M) = E_{net} \\ E_{net} > 0 \end{cases}$$

This produces: $zk_proof = Prove(statement)$

4.2.4 PoI object Construction

Upon successful verification and ZKP generation, the VVB constructs and signs the PoI object, then submits to registry.

Proof-of-Integrity (PoI) Object:

```

{
  "proof_type": "PoI",
  "inputs": {
    "project_id": "PRJ-123",
    "project_status": "REGISTERED",
    "mrv_mode": "CONTINUOUS",
    "timestamp": "2026-01-01T12:00:00Z",
    "vzb_signature": "0x7a8f...", // VVB's Critical Signature
    "methodology_hash": "H(VM0042)",
    "data_root": "0xMerkleRoot...", // Link to Step 1 data
    "serialization_range": "0-1000000", // Reserved Serial Numbers
    "amount_tCO2e": 1,
    "jurisdiction": "IN",
    "market_scope": "VCM"
  },
  "output": {
    "credit_id": "CC-789",
    "valid_from": "2026-01-01T00:00:00Z",
    "valid_until": "2026-12-31T23:59:59Z",
    "zk_proof": "0x123abc...", // Confidentiality Proof
    "cc_mint_amount": 1, // AUTHORIZED amount (not yet minted)
    "owner": "0xWalletAddress..",
    "status": "VALID"
  }
}

```

4.2.5 PoI Submission & On-Chain Verification

The VVB signed PoI is submitted to the registry:

On-Chain Action $\rightarrow$ VVB calls `submitPoI(PoI_Object)`

Registry smart contract enforces following checks:

- VVB Signature: Authenticity of `PoI.inputs.vzb_signature`.
- Duplicate Check: Has `PoI.inputs.data_root` been used before? (Prevents double-minting).
- Project Status: Is `PoI.inputs.project_status` in REGISTERED ACTIVE state?
- zk-SNARK validity: Does `Verify(zk_proof)` return True?

Validity Condition:

$$PoI_{valid} = Verify_{sign}(VVB) \wedge Verify_{zk} \wedge (data_root = R_{data}) \quad (6)$$

On Success:

PoI.status = VALIDProject_status = Active

On Failure:

Transaction reverts.

PoI.status remains NULL or REVOKED.

4.2.6 What Proof-of-Integrity (PoI) guarantees

Once PoI is accepted on-chain, the protocol guarantees following:

- Credits cannot be minted exceeding “cc_mint_amount” as it is bounded by verified reductions.
- Every future credit is mathematically linked to “data_root”.
- Only authorized VVBs can trigger the minting of credits.
- MRV integrity is enforced without data disclosure.
- ZKP prevents over-issuance of credits.

4.3 Minting & Registry Entry

This step converts the approved PoI into on-chain Carbon Credit Tokens which become traceable and ready for trade assets.

Actors Involved:

- Project Developer (D_{dev}): Triggers the minting (initial owner) transaction using valid PoI.
- Registry Smart Contract (SC_{reg}): Sole authority that mints, serializes, and record carbon credits.

Preconditions:

Requirement	Condition	Description
PoI status	VALID	Must be verified in step 2.
Project status	ACTIVE	Post PoI update
Validity windows	$T_{now} \in [T_{start}, T_{end}]$	current_time $\in$ [valid_from, valid_until].
Authorized Amount	cc_mint_amount > 0	PoI must declare a positive balance.
Serilization	Range Unused	Serials must not exist in the ledger.

Failure of any check causes the transaction to revert.

4.3.1 Mint Request (On-Chain)

The Project Developer (D_{dev}) submits mint transaction request referencing a valid PoI.

On-Chain Action $\rightarrow D_{dev}$ calls mintCredits(poi_id)

No MRV data, baseline, or raw inputs are passed here, PoI is now the single source of truth.

4.3.2 Registry Enforcement Logic

A. PoI Authentication & Authorization

The Registry Smart Contract validates the following checks:

- Verify PoI, $\text{PoI}[\text{id}].\text{owner} == \text{msg.sender}$
- Verify VVB signature
- Verify $\text{PoI}[\text{id}].\text{status} == \text{VALID}$

B. Supply Bounding (The Cap)

Let Q_{auth} be the authorized amount in the PoI:

$$Q_{\text{auth}} = \text{PoI.output.cc_mint_amount}$$

And if Q_{mint} is the requested mint amount, then registry enforces:

$$Q_{\text{mint}} \leq Q_{\text{auth}} \quad (7)$$

This makes over-issuance mathematically impossible, not just policy-restricted.

C. Serialization Locking

The `serialization_range` (e.g., 1000-2000) defined in PoI is checked (consumed / locked) against the Global Serial Registry.

- If Free: The range is marked as LOCKED.
- If Taken: Transaction reverts (prevents double-counting, replay attacks and duplicate issuance)

4.3.3 Carbon Credit Token Creation

Upon passing all checks registry mints the semi-fungible Carbon Credit Token Object.

Carbon Credit Object (Minted Asset):

```
{
  "credit_id": "#001",           // Token ID
  "project_id": "PRJ-123",      // Link to Project Object
  "vintage_year": "2025",       // Emission year (critical for pricing)
  "amount_tCO2e": 1,
  "cc_amount": 1
  "serialized_range": "0-1000000",
  "credit_type": "Removal"      // Removal / Avoidance / Emit
}
```

```

    "owner": "0xWalletAddress...",           // Current holder
    "jurisdiction": "IN",                     // Origin Country
    "market_scope": "VCM",                   // ETS / VCM
    "poi_reference": "0xPoI_Hash...",        // IMMUTABLE provenance, link to Step 2
    "expiry_date": "2032-12-31",            // Automated decay trigger
    "status": "ACTIVE"                       // ACTIVE / RETIRED / EXPIRED
  }

```

On successful minting, the registry prevents the PoI from being reused as PoI status is updated:

PoI[id].status = USED

And auditable, indexable, regulator-verifiable events are emitted:

On-Chain Action → CreditMinted (token_id, project_id, amount_tCO_{2e}, serialization_range)

4.4 Trading & Transfer

This step enables economic exchange of carbon credits without affecting credit validity, quantity, or lifecycle. It implements the “Registry-First” architecture, markets are strictly subordinate to the registry, facilitate exchange and cannot mint, modify, burn or invalidate credits.

Actors Involved:

- Credit Holder (Seller / Buyer): Entities transferring ownership of credits (e.g., Project Developers, Traders, Corporates, Individuals or Liquidity providers).
- Market & Liquidity Layer (SC_{mkt}): Pluggable smart contracts (AMMs, Order Books, OTC desks) that facilitate matching and price discovery.
- Registry Smart Contract (SC_{reg}): The underlying authority for ownership updates, validates transfers and enforces serialization rules.

Trading Preconditions:

Requirement	Condition	Description
Credit status	ACTIVE	Expired / Retired credits cannot be traded.
PoI Status	CONSUMED	Not claimed, origin PoI must be fully processed.
Expiry Check	$T_{now} < T_{expiry}$	current_time < expiry_date, auto-decay prevents trading expired assets.
Market scope	Buyer.scope == Credit.scope	Buyer/seller eligible (VCM / ETS rules)
Serilization	Range Locked & Unburned	Ensures no double-spend of specific serials.

Failure of any check causes the transaction to revert.

4.4.1 Trade Initiation (Market Interaction)

Users / Holders interact via Apps/Dashboards interfaces to initiate trades, which will support:

- AMM Swaps: Instant credit $\leftrightarrow$ stablecoin (e.g., constant product).
- Order Books: Match limit orders for specific targets.
- P2P / OTC: Direct wallet-to-wallet transfers (PAYG / micro-offset).

On-Chain Actions:

For AMM $\rightarrow$ swap(credit_id, amount, token_out)

For Direct Transfer (standard/P2P) $\rightarrow$ transferCredit(credit_id, to_address, amount)

And amount can be fractional (PAYG-ready). Markets are pure price-discovery, strictly subordinate to the Registry.

4.4.2 Registry Enforcement Logic

The Registry Smart Contract validates every swap/transfer instruction from the market layer:

- Ownership: require(msg.sender == credit.owner)
- Balance: amount $\leq$ balance, of the owner
- Expiry: Current block timestamp < expiry_date

So, Transfer Validity:

$$Transfer_{valid} = (Status == ACTIVE) \wedge (amount \leq balance) \wedge (T_{now} < T_{expiry}) \quad (8)$$

4.4.3 Post-Trade Updates

On successful validation and transfer, the registry updates the ledger state, resulting on-chain object (Buyers's View):

```
{
  "credit_id": "#001-B",           // New Batch ID if split
  "project_id": "PRJ-123",
  "owner": "0xCorporateBuyer...", // UPDATED: Ownership changed
  "poi_reference": "0xPoI_Hash...", // UNCHANGED: Provenance preserved
  "serialized_range": "50-99",     // UPDATED: Specific serials owned
  "status": "ACTIVE"
}
```

which, emits event:

On-Chain Action $\rightarrow$ CreditTransferred(credit_id, from_add, to_add, amount)

4.5 Retirements & Proof-of-Offset Issuance

This step irreversibly removes credits from circulation (burn) and issues Proof-of-Offset (PoO) the non-transferable "Settlement Receipt" that finalizes offset claims.

Actors Involved:

- Credit Holder (Claimant / Beneficiary): Corporate entity, individual, PAYG system, or compliance participant.
- Registry Smart Contract (SC_{reg}): Sole authority that enforce retirement and generates PoO.
- External Auditors / Regulators (Read-Only): Verify PoO receipts post-settlement.

Retirement Preconditions:

Requirement	Condition	Description
Credit status	ACTIVE	-
Ownership	$msg.sender == credit.owner$	Claimant controls
Expiry Check	$T_{now} < T_{expiry}$	current_time < expiry_date, no post-expiry claims
Market scope	Claim permitted (VCM / ETS)	Claimant/Beneficiary eligibility (VCM / ETS rules)
Amount	$amount \leq balance$	Cannot over-claim

Failure of any check causes the transaction to revert.

4.5.1 Retirement Request (On-Chain)

The credit holder (claimant) initiates retirement request via Apps/Dashboards interfaces,

On-Chain Action $\rightarrow$ retireCredit (credit_id, amount, claim_type, beneficiary)

Where,

- claim_type $\in$ {Corporate_NetZero, PAYG, Compliance}
- beneficiary, identifies the claiming entity

4.5.2 Registry Enforcement Logic (The Burn Gate)

The Registry smart contract executes validation and destruction checks and state updates:

- Ownership: require(msg.sender == credit.owner)
- Balance: $amount \leq balance$

- Expiry: Current block timestamp < expiry_date

4.5.3 Proof-of-Offset (PoO) Construction

After successful burn, the registry constructs a non-transferable settlement receipt/artifact.

PoO Object:

```
{
  "proof_type": "PoO",
  "inputs": {
    "project_id": "PRJ-123",           // Links to the registered Project Developer
    "credit_id": "CC-789",
    "serialization_range": "50-99" ,  // Unique IDs
    "market_scope": "VCM",           // ETS / VCM
    "cc_amount": 1
  },
  "output": {
    "burn_tx_hash": "0x998877...",    // non-double-counting proof
    "timestamp": "2026-01-01T12:00:00Z", // Time of retirement
    "beneficiary": "Corporate_Entity_X", // Who gets the credit?
    "owner": "0xWalletAddress..",      // Wallet/DID of the owner
    "claim_type": "Corporate_NetZero",  // Corporate_NetZero / PAYG / Compliance
    "zk_proof": "0x123abc...",          // Verifiable without raw data
    "amount_tCO2e": 1,                  // tCO2e authorized to be emit
    "status": "FINALIZED"               // State: REJECTED / FINALIZED
  }
}
```

PoO is non-transferable, non-revocable, and unique per retired serial range.

4.5.4 Registry State Updates & z-k claim proof (Optional)

On successful retirement, the registry updates the ledger state:

```
{
  "credit_id": "#001-B",
  "project_id": "PRJ-123",
  "owner": "0xCorporateBuyer...",    // UPDATED: Ownership changed
  "poi_reference": "0xPoO_Hash...",  //
  "serialized_range": "50-99",        // UPDATED: Specific serials owned
  "status": "RETIRED"
}
```

which, emits event:

$$\begin{aligned} \text{On-Chain Action} \rightarrow & \text{CreditRetired}(\text{credit_id}, \text{owner}, \text{amount}, \text{beneficiary}) \\ & + \\ & \text{PoOIssued}(\text{PoO_ID}, \text{credit_id}) \end{aligned}$$

This events are immutable, indexable and audit-grade.

Later, for confidentiality or sensitive reporting, the registry may assert zk claim without revealing internal consumption details.

5. Risk and Mitigations

Sr. No.	Risk & Mitigation Matrix	
	Risk	Mitigation Strategy
1	Over-crediting	PoI as a mandatory minting gate, ZKP-verified with $Q_{\text{mint}} \leq \text{cc_mint_amt}$, bound mintable supply
2	Double counting	PoI ensures single existence and PoO burn receipts single consumption
3	Hoarding	Registry tracks credit expiry/decay, stablecoin swaps and micro-offsets reduce indefinite holding.
4	Market manipulation	Registry-first hierarchy, markets can only transfer ownership, minting and burning are register-only.
5	Privacy leakage	zk-Snark-based PoI/PoO proves compliance without revealing raw data
6	Data Tampering	Partially addressed with immediate Merkle hashing of raw data streams. Future: Anomaly detection, pre-minting baseline deviation checks flag unrealistic spikes.

Table 1: Risk & Mitigation Matrix

6. Literature Review / Existing Work

Sr. No.	Comparison with existing blockchain-based carbon credit systems				
	Aspect	Prior Work	Proposed Work	Novel / Differentiated	Challenges Addressed
1	Tokenization	Fungible, semi-fungible tokenization (mint/burn), mostly credits with only mint/burn logic(e.g. ERC 20)	Serialized, non-fungible, traceable credits with expiry/decay and retirement, tracks credit lifecycle, not just balance.	Enable granular tracking with complete lifecycle management of credit.	- Fragmented implementation - Unclear life-cycle - Hoarding - High cost of origin & validity tracking
2	MRV (Auditing / Verification)	Off-Chain, relies on PDF uploads and ZKPs are used for mostly hiding wallet balances.	Mandatory MRV audits with ZKP-backed before mint.	Privacy and Trustless Issuance, handles data disclosure.	- Over-crediting - Lack of transparency
3	Explicit Proof Models	Implicit or admin base mint/burn, no "proof" artifact.	Two-Proof Model (PoI & PoO)	Cryptographic separation between legitimacy (PoI pre-min) and consumption (PoO post-retire), prevents zombie credits (retired credits being re-used).	- Double Counting - Unclear Life-cycle
4	Market Architecture	Coupled/flat, registry and market are often same.	Hierarchical, strict separation (Registry > Market)	Firewalled, registry is sole authority for existence while markets are pluggable for price discovery. Native AMMs with stablecoin swaps.	- Centralization - Environmental integrity

Table 2: Current vs Proposed Design